SESSION DE RATTRAPAGE – JEE1 ET JEE2

Code source complet

Spring Boot + Thymeleaf (HTML/CSS) + MySQL (tous les fichiers)

Clavstel El Schaddai BOYAOU

Classe : M1 GELO

TP1 : Spring Boot + HTML/CSS + MySQL

Année universitaire 2025-2026

Sommaire des fichiers

N°	Fichier	Lignes
1	README.md	20
2	pom.xml	40
3	src/main/resources/application.properties	15
4	src/main/resources/messages.properties	1
5	database/facturation.sql	25
6	src/main/java/com/facturation/FacturationApplication.java	11
7	src/main/java/com/facturation/model/Client.java	40
8	src/main/java/com/facturation/model/Facture.java	40
9	src/main/java/com/facturation/model/Gerant.java	33
10	src/main/java/com/facturation/repository/ClientRepository.java	11
11	src/main/java/com/facturation/repository/FactureRepository.java	16
12	src/main/java/com/facturation/repository/GerantRepository.java	10
13	src/main/java/com/facturation/security/GerantUserDetailsService.java	25
14	src/main/java/com/facturation/config/DataInitializer.java	39
15	src/main/java/com/facturation/config/SecurityConfig.java	32
16	src/main/java/com/facturation/controller/ClientController.java	76
17	src/main/java/com/facturation/controller/FactureController.java	88
18	src/main/java/com/facturation/controller/LoginController.java	18
19	src/main/resources/templates/fragments/layout.html	19
20	src/main/resources/templates/login.html	29
21	src/main/resources/templates/clients.html	45
22	src/main/resources/templates/client-form.html	33
23	src/main/resources/templates/factures.html	46
24	src/main/resources/templates/facture-form.html	40
25	src/main/resources/static/css/style.css	47

1. README.md

```
# TP1 JEE - Application CRUD de facturation (Spring Boot + HTML/CSS + MySQL)

Auteur : Clavstel El Schaddai BOYAOU - M1 GELO

## Prerequis
- Java 17+ et Maven 3.9+
- MySQL 8 demarre en local (port 3306)

## Lancement
1. Verifier `src/main/resources/application.properties` (utilisateur / mot de passe MySQL).
   La base `facturation_db` et les tables sont creees automatiquement.
   (Script equivalent facultatif : `database/facturation.sql`)
2. mvn spring-boot:run
3. Ouvrir http://localhost:8080

## Compte gerant (cree au premier demarrage)
Login : admin   -   Mot de passe : admin123

## Fonctionnalites
Authentification du gerant ; ajout, modification, suppression et recherche des clients et des
      factures.
```

2. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
      instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
      https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>
    <groupId>com.facturation</groupId>
    <artifactId>facturation-tp1</artifactId>
    <version>1.0.0</version>
    <name>facturation-tp1</name>
    <description>TP1 JEE - Application CRUD de facturation (Spring Boot + Thymeleaf +
      MySQL)</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
      web</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
      thymeleaf</artifactId></dependency>
        <dependency><groupId>org.thymeleaf.extras</groupId><artifactId>thymeleaf-extras-
      springsecurity6</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
      jpa</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
      security</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
      validation</artifactId></dependency>
        <dependency><groupId>com.mysql</groupId><artifactId>mysql-
      connector-j</artifactId><scope>runtime</scope></dependency>
        <dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>tru
      e</optional></dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes><exclude><groupId>org.projectlombok</groupId><artifactId>lombok</artif
      actId></exclude></excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
```

```
</project>
```

3. src/main/resources/application.properties

```
spring.application.name=facturation-tp1
server.port=8080

# --- MySQL : adapter l'utilisateur et le mot de passe a votre installation ---
spring.datasource.url=jdbc:mysql://localhost:3306/facturation_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=

# --- JPA / Hibernate : cree et met a jour les tables automatiquement ---
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.jpa.show-sql=false

# --- Thymeleaf ---
spring.thymeleaf.cache=false
```

4. src/main/resources/messages.properties

```
typeMismatch.montant=Veuillez saisir un nombre valide
```

5. database/facturation.sql

```
-- Script MySQL (facultatif : Hibernate cree deja ces tables avec ddl-auto=update)
CREATE DATABASE IF NOT EXISTS facturation_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE facturation_db;

CREATE TABLE IF NOT EXISTS gerant (
    id       BIGINT AUTO_INCREMENT PRIMARY KEY,
    nom      VARCHAR(100) NOT NULL,
    login    VARCHAR(50)  NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL            -- hash BCrypt
);

CREATE TABLE IF NOT EXISTS client (
    idclient BIGINT AUTO_INCREMENT PRIMARY KEY,
    nom      VARCHAR(100) NOT NULL,
    tel      VARCHAR(20)
);

CREATE TABLE IF NOT EXISTS facture (
    idfact   BIGINT AUTO_INCREMENT PRIMARY KEY,
    montant  DOUBLE       NOT NULL,
    objet    VARCHAR(255) NOT NULL,
    idclient BIGINT       NOT NULL,
    CONSTRAINT fk_facture_client FOREIGN KEY (idclient)
        REFERENCES client (idclient) ON DELETE CASCADE
);
```

6. src/main/java/com/facturation/FacturationApplication.java

```
package com.facturation;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FacturationApplication {
    public static void main(String[] args) {
        SpringApplication.run(FacturationApplication.class, args);
    }
}
```

7. src/main/java/com/facturation/model/Client.java

```
package com.facturation.model;

import jakarta.persistence.*;
```

```java
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.util.ArrayList;
import java.util.List;

/** Client(Idclient, nom, tel). Un client possede plusieurs factures. */
@Entity
@Table(name = "client")
@Getter @Setter @NoArgsConstructor
public class Client {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "idclient")
    private Long idClient;

    @NotBlank(message = "Le nom est obligatoire")
    @Column(nullable = false, length = 100)
    private String nom;

    @Pattern(regexp = "^$|^[0-9+ ]{6,20}$", message = "Numéro de téléphone invalide")
    @Column(length = 20)
    private String tel;

    // Suppression d'un client => suppression de ses factures
    @OneToMany(mappedBy = "client", cascade = CascadeType.REMOVE)
    private List<Facture> factures = new ArrayList<>();

    public Client(String nom, String tel) {
        this.nom = nom;
        this.tel = tel;
    }
}
```

8. src/main/java/com/facturation/model/Facture.java

```java
package com.facturation.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

/** Facture(Idfact, montant, objet, IdClient). IdClient est la cle etrangere vers Client. */
@Entity
@Table(name = "facture")
@Getter @Setter @NoArgsConstructor
public class Facture {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "idfact")
    private Long idFact;

    @NotNull(message = "Le montant est obligatoire")
    @Positive(message = "Le montant doit être positif")
    @Column(nullable = false)
    private Double montant;

    @NotBlank(message = "L'objet est obligatoire")
    @Column(nullable = false)
    private String objet;

    @ManyToOne(optional = false)
    @JoinColumn(name = "idclient", nullable = false)
    private Client client;

    public Facture(Double montant, String objet, Client client) {
        this.montant = montant;
```

```
        this.objet = objet;
        this.client = client;
    }
}
```

9. src/main/java/com/facturation/model/Gerant.java

```
package com.facturation.model;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

/** Gerant(Id, nom, login, password) - mot de passe stocke chiffre (BCrypt). */
@Entity
@Table(name = "gerant")
@Getter @Setter @NoArgsConstructor
public class Gerant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @Column(nullable = false, length = 100)
    private String nom;

    @Column(nullable = false, unique = true, length = 50)
    private String login;

    @Column(nullable = false, length = 100)
    private String password;

    public Gerant(String nom, String login, String password) {
        this.nom = nom;
        this.login = login;
        this.password = password;
    }
}
```

10. src/main/java/com/facturation/repository/ClientRepository.java

```
package com.facturation.repository;

import com.facturation.model.Client;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface ClientRepository extends JpaRepository<Client, Long> {
    // Recherche par nom (sans tenir compte de la casse) ou par telephone
    List<Client> findByNomContainingIgnoreCaseOrTelContaining(String nom, String tel);
}
```

11. src/main/java/com/facturation/repository/FactureRepository.java

```
package com.facturation.repository;

import com.facturation.model.Facture;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;

public interface FactureRepository extends JpaRepository<Facture, Long> {

    // Recherche par objet ou par nom du client
    @Query("select f from Facture f where lower(f.objet) like lower(concat('%', :q, '%')) "
         + "or lower(f.client.nom) like lower(concat('%', :q, '%')) order by f.idFact desc")
    List<Facture> search(@Param("q") String q);
}
```

12. src/main/java/com/facturation/repository/GerantRepository.java

```java
package com.facturation.repository;

import com.facturation.model.Gerant;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface GerantRepository extends JpaRepository<Gerant, Long> {
    Optional<Gerant> findByLogin(String login);
}
```

13. src/main/java/com/facturation/security/GerantUserDetailsService.java

```java
package com.facturation.security;

import com.facturation.model.Gerant;
import com.facturation.repository.GerantRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

/** Fournit a Spring Security le gerant correspondant au login saisi. */
@Service
@RequiredArgsConstructor
public class GerantUserDetailsService implements UserDetailsService {

    private final GerantRepository gerants;

    @Override
    public UserDetails loadUserByUsername(String login) throws UsernameNotFoundException {
        Gerant g = gerants.findByLogin(login)
                .orElseThrow(() -> new UsernameNotFoundException("Gérant introuvable : " + login));
        return User.withUsername(g.getLogin()).password(g.getPassword()).roles("GERANT").build();
    }
}
```

14. src/main/java/com/facturation/config/DataInitializer.java

```java
package com.facturation.config;

import com.facturation.model.Client;
import com.facturation.model.Facture;
import com.facturation.model.Gerant;
import com.facturation.repository.ClientRepository;
import com.facturation.repository.FactureRepository;
import com.facturation.repository.GerantRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

/** Cree le gerant par defaut et des donnees de demonstration si la base est vide. */
@Component
@RequiredArgsConstructor
public class DataInitializer implements CommandLineRunner {

    private final GerantRepository gerants;
    private final ClientRepository clients;
    private final FactureRepository factures;
    private final PasswordEncoder encoder;

    @Override
    public void run(String... args) {
        if (gerants.count() == 0) {
            gerants.save(new Gerant("Administrateur", "admin", encoder.encode("admin123")));
        }
        if (clients.count() == 0) {
            Client a = clients.save(new Client("Moussa Diop", "771234567"));
            Client b = clients.save(new Client("Awa Ndiaye", "781112233"));
```

```
            Client c = clients.save(new Client("Cheikh Fall", "765554433"));
            factures.save(new Facture(150000.0, "Achat de materiel informatique", a));
            factures.save(new Facture(45000.0, "Maintenance reseau", a));
            factures.save(new Facture(98500.0, "Licence logicielle annuelle", b));
            factures.save(new Facture(27500.0, "Formation utilisateurs", c));
        }
    }
}
```

15. src/main/java/com/facturation/config/SecurityConfig.java

```
package com.facturation.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
                    .requestMatchers("/login", "/css/**").permitAll()
                    .anyRequest().hasRole("GERANT"))        // tout le reste : gerant authentifie
            .formLogin(form -> form
                    .loginPage("/login")
                    .defaultSuccessUrl("/clients", true)
                    .permitAll())
            .logout(logout -> logout
                    .logoutSuccessUrl("/login?logout")
                    .permitAll());
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

16. src/main/java/com/facturation/controller/ClientController.java

```
package com.facturation.controller;

import com.facturation.model.Client;
import com.facturation.repository.ClientRepository;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

/** CRUD + recherche des clients. */
@Controller
@RequestMapping("/clients")
@RequiredArgsConstructor
public class ClientController {

    private final ClientRepository clients;

    @GetMapping
    public String list(@RequestParam(defaultValue = "") String q, Model model) {
        String term = q.trim();
        model.addAttribute("clients", term.isEmpty()
                ? clients.findAll(Sort.by("nom"))
                : clients.findByNomContainingIgnoreCaseOrTelContaining(term, term));
        model.addAttribute("q", q);
        return "clients";
```

```
    }

    @GetMapping("/nouveau")
    public String nouveau(Model model) {
        model.addAttribute("client", new Client());
        return "client-form";
    }

    @GetMapping("/{id}/modifier")
    public String modifier(@PathVariable Long id, Model model, RedirectAttributes ra) {
        return clients.findById(id).map(c -> {
            model.addAttribute("client", c);
            return "client-form";
        }).orElseGet(() -> {
            ra.addFlashAttribute("error", "Client introuvable");
            return "redirect:/clients";
        });
    }

    @PostMapping("/save")
    public String save(@Valid @ModelAttribute("client") Client form, BindingResult result,
                       RedirectAttributes ra) {
        if (result.hasErrors()) {
            return "client-form";
        }
        boolean creation = form.getIdClient() == null;
        Client client = creation ? form : clients.findById(form.getIdClient()).orElseThrow();
        if (!creation) {            // on ne recopie que les champs modifiables
            client.setNom(form.getNom());
            client.setTel(form.getTel());
        }
        clients.save(client);
        ra.addFlashAttribute("success", creation ? "Client ajouté avec succès" : "Client modifié
      avec succès");
        return "redirect:/clients";
    }

    @PostMapping("/{id}/supprimer")
    public String supprimer(@PathVariable Long id, RedirectAttributes ra) {
        if (clients.existsById(id)) {
            clients.deleteById(id);      // supprime aussi ses factures (cascade)
            ra.addFlashAttribute("success", "Client et factures associées supprimés");
        } else {
            ra.addFlashAttribute("error", "Client introuvable");
        }
        return "redirect:/clients";
    }
}
```

17. src/main/java/com/facturation/controller/FactureController.java

```
package com.facturation.controller;

import com.facturation.model.Client;
import com.facturation.model.Facture;
import com.facturation.repository.ClientRepository;
import com.facturation.repository.FactureRepository;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

/** CRUD + recherche des factures. */
@Controller
@RequestMapping("/factures")
@RequiredArgsConstructor
public class FactureController {

    private final FactureRepository factures;
    private final ClientRepository clients;

    @GetMapping
```

```
26      public String list(@RequestParam(defaultValue = "") String q, Model model) {
27          String term = q.trim();
28          model.addAttribute("factures", term.isEmpty()
29                  ? factures.findAll(Sort.by(Sort.Direction.DESC, "idFact"))
30                  : factures.search(term));
31          model.addAttribute("q", q);
32          return "factures";
33      }
34
35      @GetMapping("/nouveau")
36      public String nouveau(Model model) {
37          Facture f = new Facture();
38          f.setClient(new Client());
39          return formulaire(f, model);
40      }
41
42      @GetMapping("/{id}/modifier")
43      public String modifier(@PathVariable Long id, Model model, RedirectAttributes ra) {
44          return factures.findById(id).map(f -> formulaire(f, model)).orElseGet(() -> {
45              ra.addFlashAttribute("error", "Facture introuvable");
46              return "redirect:/factures";
47          });
48      }
49
50      @PostMapping("/save")
51      public String save(@Valid @ModelAttribute("facture") Facture form, BindingResult result,
52                         Model model, RedirectAttributes ra) {
53          Long clientId = form.getClient() != null ? form.getClient().getIdClient() : null;
54          Client client = clientId == null ? null : clients.findById(clientId).orElse(null);
55          if (client == null) {
56              result.rejectValue("client", "client.required", "Veuillez choisir un client");
57          }
58          if (result.hasErrors()) {
59              if (form.getClient() == null) form.setClient(new Client());
60              return formulaire(form, model);
61          }
62          boolean creation = form.getIdFact() == null;
63          Facture facture = creation ? form : factures.findById(form.getIdFact()).orElseThrow();
64          facture.setMontant(form.getMontant());
65          facture.setObjet(form.getObjet());
66          facture.setClient(client);
67          factures.save(facture);
68          ra.addFlashAttribute("success", creation ? "Facture ajoutée avec succès" : "Facture modifiée
            avec succès");
69          return "redirect:/factures";
70      }
71
72      @PostMapping("/{id}/supprimer")
73      public String supprimer(@PathVariable Long id, RedirectAttributes ra) {
74          if (factures.existsById(id)) {
75              factures.deleteById(id);
76              ra.addFlashAttribute("success", "Facture supprimée");
77          } else {
78              ra.addFlashAttribute("error", "Facture introuvable");
79          }
80          return "redirect:/factures";
81      }
82
83      private String formulaire(Facture facture, Model model) {
84          model.addAttribute("facture", facture);
85          model.addAttribute("clients", clients.findAll(Sort.by("nom")));
86          return "facture-form";
87      }
88  }
```

18. src/main/java/com/facturation/controller/LoginController.java

```
1   package com.facturation.controller;
2
3   import org.springframework.stereotype.Controller;
4   import org.springframework.web.bind.annotation.GetMapping;
5
6   @Controller
7   public class LoginController {
8
9       @GetMapping("/login")
```

```
    public String login() {
        return "login";
    }

    @GetMapping("/")
    public String home() {
        return "redirect:/clients";
    }
}
```

19. src/main/resources/templates/fragments/layout.html

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<body>
<!-- Barre de navigation commune a toutes les pages -->
<header th:fragment="navbar(active)" class="navbar">
    <div class="brand">Facturation</div>
    <nav>
        <a th:href="@{/clients}" th:classappend="${active == 'clients'} ? 'active'">Clients</a>
        <a th:href="@{/factures}" th:classappend="${active == 'factures'} ? 'active'">Factures</a>
    </nav>
    <div class="user">
        <span><span sec:authentication="name">admin</span> (Gérant)</span>
        <form th:action="@{/logout}" method="post" class="inline">
            <button class="btn small secondary" type="submit">Déconnexion</button>
        </form>
    </div>
</header>
</body>
</html>
```

20. src/main/resources/templates/login.html

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Connexion - Facturation</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div class="login-wrap">
    <div class="card">
        <h1>Connexion</h1>
        <p class="muted">Espace gérant - Gestion de la facturation</p>
        <div class="error" th:if="${param.error}">Login ou mot de passe incorrect</div>
        <div class="success" th:if="${param.logout}">Vous avez été déconnecté</div>
        <form th:action="@{/login}" method="post">
            <div class="field full">
                <label for="username">Login</label>
                <input type="text" id="username" name="username" autofocus required>
            </div>
            <div class="field full">
                <label for="password">Mot de passe</label>
                <input type="password" id="password" name="password" required>
            </div>
            <button class="btn full-btn" type="submit">Se connecter</button>
        </form>
    </div>
</div>
</body>
</html>
```

21. src/main/resources/templates/clients.html

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Clients - Facturation</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
```

```
<body>
<div th:replace="~{fragments/layout :: navbar('clients')}"></div>
<main class="container">
    <h1>Gestion des clients</h1>
    <div class="success" th:if="${success}" th:text="${success}"></div>
    <div class="error" th:if="${error}" th:text="${error}"></div>

    <div class="card">
        <div class="toolbar">
            <form th:action="@{/clients}" method="get">
                <input type="text" name="q" th:value="${q}" placeholder="Rechercher par nom ou téléphone...">
            </form>
            <a class="btn" th:href="@{/clients/nouveau}">+ Nouveau client</a>
        </div>
        <table>
            <thead>
            <tr><th>N°</th><th>Nom</th><th>Téléphone</th><th style="width:170px">Actions</th></tr>
            </thead>
            <tbody>
            <tr th:if="${#lists.isEmpty(clients)}"><td colspan="4" class="muted">Aucun client trouvé.</td></tr>
            <tr th:each="c : ${clients}">
                <td th:text="${c.idClient}">1</td>
                <td th:text="${c.nom}">Nom</td>
                <td th:text="${c.tel}">770000000</td>
                <td>
                    <a class="btn small" th:href="@{/clients/{id}/modifier(id=${c.idClient})}">Modifier</a>
                    <form class="inline" method="post" th:action="@{/clients/{id}/supprimer(id=${c.idClient})}"
                          onsubmit="return confirm('Supprimer ce client et toutes ses factures ?');">
                        <button class="btn small danger" type="submit">Supprimer</button>
                    </form>
                </td>
            </tr>
            </tbody>
        </table>
    </div>
</main>
</body>
</html>
```

22. src/main/resources/templates/client-form.html

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Client - Facturation</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div th:replace="~{fragments/layout :: navbar('clients')}"></div>
<main class="container">
    <h1>Gestion des clients</h1>
    <div class="card">
        <h2 th:text="${client.idClient == null} ? 'Nouveau client' : 'Modifier le client'">Nouveau client</h2>
        <form th:action="@{/clients/save}" th:object="${client}" method="post">
            <input type="hidden" th:field="*{idClient}">
            <div class="field half">
                <label for="nom">Nom</label>
                <input type="text" id="nom" th:field="*{nom}">
                <div class="field-error" th:if="${#fields.hasErrors('nom')}" th:errors="*{nom}"></div>
            </div>
            <div class="field half">
                <label for="tel">Téléphone</label>
                <input type="text" id="tel" th:field="*{tel}">
                <div class="field-error" th:if="${#fields.hasErrors('tel')}" th:errors="*{tel}"></div>
            </div>
            <br>
            <button class="btn" type="submit">Enregistrer</button>
```

```
            <a class="btn secondary" th:href="@{/clients}">Annuler</a>
        </form>
    </div>
</main>
</body>
</html>
```

23. src/main/resources/templates/factures.html

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Factures - Facturation</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div th:replace="~{fragments/layout :: navbar('factures')}"></div>
<main class="container">
    <h1>Gestion des factures</h1>
    <div class="success" th:if="${success}" th:text="${success}"></div>
    <div class="error" th:if="${error}" th:text="${error}"></div>

    <div class="card">
        <div class="toolbar">
            <form th:action="@{/factures}" method="get">
                <input type="text" name="q" th:value="${q}" placeholder="Rechercher par objet ou nom du client...">
            </form>
            <a class="btn" th:href="@{/factures/nouveau}">+ Nouvelle facture</a>
        </div>
        <table>
            <thead>
            <tr><th>N°</th><th>Client</th><th>Objet</th><th>Montant</th><th style="width:170px">Actions</th></tr>
            </thead>
            <tbody>
            <tr th:if="${#lists.isEmpty(factures)}"><td colspan="5" class="muted">Aucune facture trouvée.</td></tr>
            <tr th:each="f : ${factures}">
                <td th:text="${f.idFact}">1</td>
                <td th:text="${f.client.nom}">Client</td>
                <td th:text="${f.objet}">Objet</td>
                <td class="num" th:text="${#numbers.formatDecimal(f.montant, 1, 'WHITESPACE', 0, 'POINT')} + ' FCFA'">0 FCFA</td>
                <td>
                    <a class="btn small" th:href="@{/factures/{id}/modifier(id=${f.idFact})}">Modifier</a>
                    <form class="inline" method="post" th:action="@{/factures/{id}/supprimer(id=${f.idFact})}"
                          onsubmit="return confirm('Supprimer cette facture ?');">
                        <button class="btn small danger" type="submit">Supprimer</button>
                    </form>
                </td>
            </tr>
            </tbody>
        </table>
    </div>
</main>
</body>
</html>
```

24. src/main/resources/templates/facture-form.html

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Facture - Facturation</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div th:replace="~{fragments/layout :: navbar('factures')}"></div>
<main class="container">
    <h1>Gestion des factures</h1>
```

```
    <div class="card">
        <h2 th:text="${facture.idFact == null} ? 'Nouvelle facture' : 'Modifier la facture'">Nouvelle facture</h2>
      <form th:action="@{/factures/save}" th:object="${facture}" method="post">
          <input type="hidden" th:field="*{idFact}">
          <div class="field half">
              <label for="client">Client</label>
              <select id="client" th:field="*{client.idClient}">
                  <option value="">-- Choisir un client --</option>
                  <option th:each="c : ${clients}" th:value="${c.idClient}" th:text="${c.nom}">Client</option>
              </select>
              <div class="field-error" th:if="${#fields.hasErrors('client')}" th:errors="*{client}"></div>
          </div>
          <div class="field half">
              <label for="montant">Montant (FCFA)</label>
              <input type="text" id="montant" th:field="*{montant}">
              <div class="field-error" th:if="${#fields.hasErrors('montant')}" th:errors="*{montant}"></div>
          </div>
          <div class="field full">
              <label for="objet">Objet</label>
              <input type="text" id="objet" th:field="*{objet}">
              <div class="field-error" th:if="${#fields.hasErrors('objet')}" th:errors="*{objet}"></div>
          </div>
          <button class="btn" type="submit">Enregistrer</button>
          <a class="btn secondary" th:href="@{/factures}">Annuler</a>
      </form>
    </div>
</main>
</body>
</html>
```

25. src/main/resources/static/css/style.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #f1f5f9; color: #1e293b; }
h1 { font-size: 24px; margin: 0 0 18px; }
h2 { font-size: 18px; margin: 0 0 14px; }
.muted { color: #64748b; font-size: 14px; }

.navbar { background: #1e3a8a; color: #fff; padding: 14px 28px; overflow: hidden; }
.navbar .brand { float: left; font-size: 20px; font-weight: bold; padding-top: 4px; }
.navbar nav { float: left; margin-left: 30px; }
.navbar nav a { color: #cbd5e1; text-decoration: none; margin-right: 8px; padding: 8px 14px; border-radius: 6px; display: inline-block; }
.navbar nav a.active { background: #2563eb; color: #fff; }
.navbar .user { float: right; padding-top: 3px; font-size: 14px; }
.navbar .user span { margin-right: 12px; }

.container { max-width: 1000px; margin: 28px auto; padding: 0 20px; }
.card { background: #fff; border-radius: 10px; padding: 22px; box-shadow: 0 1px 4px rgba(0,0,0,.12); margin-bottom: 22px; }

input, select { padding: 9px 11px; border: 1px solid #cbd5e1; border-radius: 6px; font-size: 14px; width: 100%; background: #fff; }
.field { margin-bottom: 14px; vertical-align: top; display: inline-block; }
.field.full { display: block; }
.field.half { width: 49%; padding-right: 12px; }
.field label { display: block; font-size: 13px; font-weight: bold; margin-bottom: 5px; }

.btn { background: #2563eb; color: #fff; border: 0; padding: 10px 18px; border-radius: 6px; cursor: pointer; font-size: 14px; }
.btn.secondary { background: #64748b; }
.btn.danger { background: #dc2626; }
.btn.small { padding: 6px 11px; font-size: 13px; }
.full-btn { width: 100%; }

.toolbar { overflow: hidden; margin-bottom: 16px; }
.toolbar form { float: left; width: 62%; }
.toolbar > .btn { float: right; }

table { width: 100%; border-collapse: collapse; }
th { background: #e2e8f0; text-align: left; padding: 10px; font-size: 13px; }
```

```
td { padding: 10px; border-bottom: 1px solid #e2e8f0; font-size: 14px; }
td.num { white-space: nowrap; font-weight: bold; }
.total { text-align: right; margin-top: 16px; font-size: 16px; }

.error { background: #fee2e2; color: #991b1b; padding: 10px 14px; border-radius: 6px; margin-bottom:
       16px; font-size: 14px; }
.login-wrap { width: 400px; margin: 90px auto; }

/* --- ajouts TP1 (pages HTML serveur) --- */
a.btn { text-decoration: none; display: inline-block; }
.success { background: #dcfce7; color: #166534; padding: 10px 14px; border-radius: 6px; margin-
      bottom: 16px; font-size: 14px; }
.field-error { color: #b91c1c; font-size: 12px; margin-top: 4px; }
.inline { display: inline; }
```